CM11 - Strings, I/O, Files

String library, bounded input, logical files, file read/write

Louis Ledoux

2026-07-05

Input/Output and Strings

Input/Output and Strings

Live pacing: about 14 min

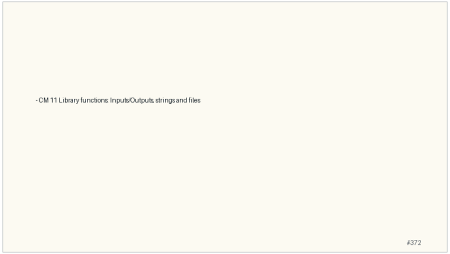
Question. When does an array behave like a pointer, and when does it not?

Core claim. Arrays are contiguous objects; many expressions expose the address of their first element.

Target capability. Students can draw array storage, pointer arithmetic, strings, and 2D indexing.

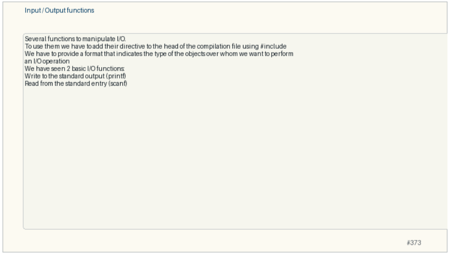
Main Path

- Start from the question: When does an array behave like a pointer, and when does it not?
- Build the model: Arrays are contiguous objects; many expressions expose the address of their first element.
- End with a checkable action: Students can draw array storage, pointer arithmetic, strings, and 2D indexing.
- Keep only this slide on the horizontal path during the live lecture.



Worked Example

- Use this as the live trace: Use `arr`, `&arr[0]`, `arr + 1`, then repeat with a string ending in `\0`.
- Representative source slide: 373
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.



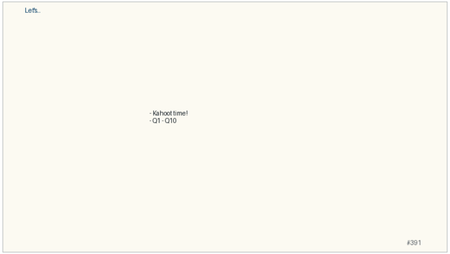
Anecdote Or Motivation

A string is not magic in C; it is an array convention plus library functions that respect it.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.



Backup Index

Original slides 372-391

- CM 11 Library functions: Inputs/Outputs, strings and files

Backup: CM 11 Library functions: Inputs/Outputs, strings and files

Original slide 372

- CM 11 Library functions: Inputs/Outputs, strings and files

Backup: Input / Output functions

Original slide 373

```
Several functions to manipulate I/O.
To use them we have to add their directive to the h                 n fi
We have to provide a format that indicates the type                 whom
We have seen 2 basic I/O functions:
Write to the standard output (printf)
Read from the standard entry (scanf)
```

Input / Output functions

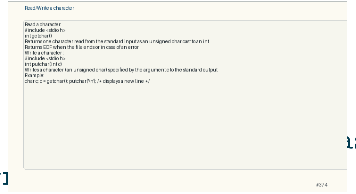
Several functions to manipulate I/O.
To use them we have to add their directive to the head of the compilation file using #include
We have to provide a format that indicates the type of the objects over whom we want to perform
an I/O operation
We have seen 2 basic I/O functions:
Write to the standard output (printf)
Read from the standard entry (scanf)

#373

Backup: Read/Write a character

Original slide 374

```
Read a character:
#include <stdio.h>
int getchar()
Returns one character read from the standard input                           ast
Returns EOF when the file ends or in case of an err
Write a character :
#include <stdio.h>
int putchar(int c)
Writes a character (an unsigned char) specified by the argument c to the sta
Example:
char c; c = getchar(); putchar('\n'); /* displays a new line */
```


Read/Write a character

```
Read a character:
#include <stdio.h>
int getchar()
Returns one character read from the standard input as an unsigned char cast to an int
Returns EOF when the file ends or in case of an error
Write a character :
#include <stdio.h>
int putchar(int c)
Writes a character (an unsigned char) specified by the argument c to the standard output
Example:
char c; c = getchar(); putchar('\n'); /* displays a new line */
```

#374

Backup: String manipulation

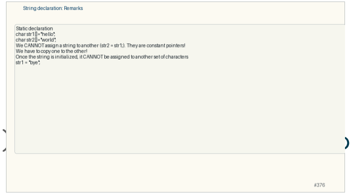
Original slide 375

```
#include<string.h>
Read and write strings: scanf(...)/printf(...),
gets(...)/puts(...),
fgets(...)/fputs(...), ...
Length of string : strlen(...), ...
Comparison of strings : strcmp(...), ...
Copy of strings : strcpy(...), ...
Formatted output to a string: sprintf(...), ...
...
```

Backup: String declaration: Remarks

Original slide 376

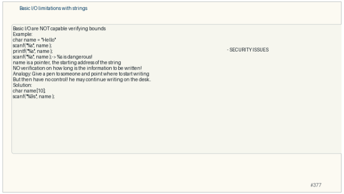
```
Static declaration
char str1[]="hello";
char str2[]="world";
We CANNOT assign a string to another (str2 = str1;)
We have to copy one to the other!
Once the string is initialized, it CANNOT be assigned to another set of char
str1 = "bye";
```



Backup: Basic I/O limitations with strings

Original slide 377

```
Basic I/O are NOT capable verifying bounds
Example:
char name = "Hello"
scanf("%s", name );
printf("%s", name );
scanf("%s", name );-> %s is dangerous!
name is a pointer, the starting address of the string
NO verification on how long is the information to be written!
Analogy: Give a pen to someone and point where to start writing
But then have no control! he may continue writing on the desk..
Solution:
char name[10];
scanf("%9s", name );
```



- SECURITY ISSUES

Backup: Basic I/O limitations with strings

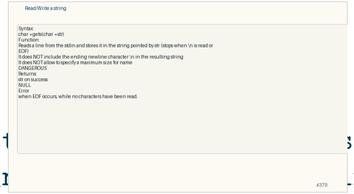
Original slide 378

```
scanf ends after a space:
receiving multiword strings in one variable
Example
char name = "Hello John"
scanf("%s", name );
printf("%s", name );
Solution?
gets(name);
puts(name);
```

Backup: Read/Write a string

Original slide 379

```
Syntax:
char *gets(char *str)
Function:
Reads a line from the stdin and stores it in the st                                stop
It does NOT include the ending newline character \r                                ing
It does NOT allow to specify a maximum size for name
DANGEROUS
Returns:
str on success
NULL
Error
when EOF occurs, while no characters have been read.
```


Read/Write a string

Syntax:
char *gets(char *str)
Function:
Reads a line from the stdin and stores it in the string pointed by str (stops when \n is read or
EOF)
It does NOT include the ending newline character \n in the resulting string
It does NOT allow to specify a maximum size for name
DANGEROUS
Returns:
str on success
NULL
Error
when EOF occurs, while no characters have been read.

#379

Backup: Read/Write a string

Original slide 380

```
Syntax:
char *fgets (char *str, int size, FILE* file);
Function:
Reads size - 1 characters from input stream file an                 ring
Stops whichever happens first:
size - 1 characters have been read
A newline occurs, \n
The end-of-file is reached, EOF
It includes new line character `\n` pressed by the user
A terminating null character `\0'is automatically appended after the charact
Returns:
str on success
NULL :Error or when EOF occurs, while no characters have been read.
```

Backup: Read/Write a string

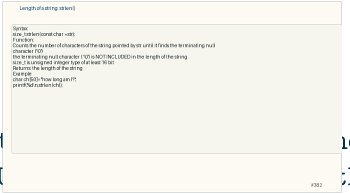
Original slide 381

```
int sscanf (const char *str, const char * format,
Like scanf but reads data from string pointed by st                    ies
And much other functions, check libraries documenta
```

Backup: Length of a string: strlen()

Original slide 382

```
Syntax:
size_t strlen(const char *str);
Function:
Counts the number of characters of the string point                nds
the terminating null character ( '\0') is NOT INCLU                the
size_t is unsigned integer type of at least 16 bit
Returns: the length of the string
Example
char ch[50]="how long am I?";
printf(%d\n,strlen(ch));
```



Length of a string: strlen()

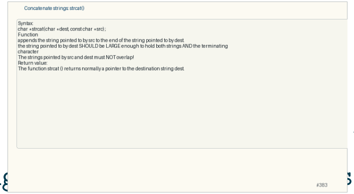
Syntax:
size_t strlen(const char *str);
Function:
Counts the number of characters of the string pointed by str until it finds the terminating null
character ('\0')
the terminating null character ('\0') is NOT INCLUDED in the length of the string
size_t is unsigned integer type of at least 16 bit
Returns: the length of the string
Example
char ch[50]="how long am I?";
printf(%d\n,strlen(ch));

#382

Backup: Concatenate strings: strcat()

Original slide 383

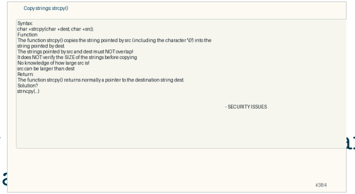
```
Syntax:
char *strcat(char *dest, const char *src) ;
Function
appends the string pointed to by src to the end of                    by
the string pointed to by dest SHOULD be LARGE enoug                  s AN
The strings pointed by src and dest must NOT overlap!
Return value:
The function strcat () returns normally a pointer to the destination string
```



Backup: Copy strings: strcpy()

Original slide 384

```
Syntax:
char *strcpy(char *dest, char *src);
Function
The function strcpy() copies the string pointed by                 arac
The strings pointed by src and dest must NOT overla
It does NOT verify the SIZE of the strings before copying
No knowledge of how large src is!
src can be larger than dest
Return:
The function strcpy() returns normally a pointer to the destination string d
Solution?
strncpy(...)
```

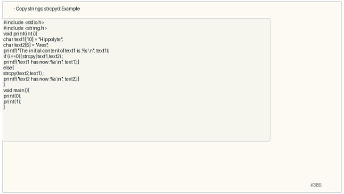


- SECURITY ISSUES

Backup: Copy strings: strcpy():Example

Original slide 385

```c
#include <stdio.h>
#include <string.h>
void print(int i){
char text1[10] = "Hippolyte";
char text2[5] = "Ares";
printf("The initial content of text1 is :%s \n", text1);
if (i==0){ strcpy(text1,text2) ;
printf("text1 has now :%s \n", text1);}
else{
strcpy(text2,text1) ;
printf("text2 has now :%s \n", text2);}
}
void main(){
print(0);
print(1);
```



Backup: Copy strings: strncpy()

Original slide 386

```
Syntax:
char *strncpy(char *dest, char *src, size_t n)
Function:
Copies UP TO to n characters from the string pointe
Length of src < n -> the remaining part of dest is s.
Length of src > n -> no termination character at dest
Return:
The function strncpy() returns normally a pointer to the destination string
```


Copy strings: strncpy()
Syntax:
char *strncpy(char *dest, char *src, size_t n)
Function:
Copies UP TO to n characters from the string pointed by src to dest.
Length of src < n -> the remaining part of dest is padded with null bytes.
Length of src > n -> no termination character at dest
Return:
The function strncpy() returns normally a pointer to the destination string dest.
#386

Backup: Compare strings: strcmp()

Original slide 387

```
Syntax:
char strcmp (char *str1, char *str2);
Function
The function strcmp() compares two strings of point
Return: The function strcmp() returns an integer:
if Return value < 0 then it indicates str1 is less than str2.
if Return value > 0 then it indicates str2 is less than str1.
if Return value = 0 then it indicates str1 is equal to str2.
```

Backup: Compare strings: strncmp()

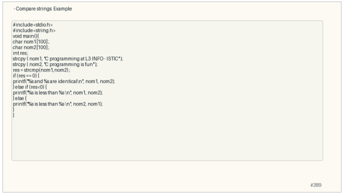
Original slide 388

```
Syntax:
int strncmp(const char *str1, const char *str2, siz
Function
The function strncmp() compares at most the first r                              bet
Return value: The function strncmp() returns an int
if Return value < 0 then it indicates str1 is less than str2.
if Return value > 0 then it indicates str2 is less than str1.
if Return value = 0 then it indicates str1 is equal to str2.
```

Backup: Compare strings: Example

Original slide 389

```c
#include<stdio.h>
#include<string.h>
void main(){
char nom1[100] ;
char nom2[100] ;
int res ;
strcpy ( nom1, "C programming at L3 INFO - ISTIC" );
strcpy ( nom2, "C programming is fun" );
res = strcmp(nom1,nom2) ;
if (res == 0) {
printf("%s and %s are identical\n", nom1, nom2);
} else if (res<0) {
printf("%s is less than %s \n", nom1, nom2);
} else {
printf("%s is less than %s \n", nom2, nom1);
```



Backup: Compare strings: Example

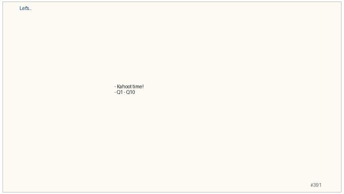
Original slide 390

```c
#include <stdio.h>
#include <string.h>
int main(){
char s1[20];
fgets(s1, 20, stdin);
strcmp(s1, "login");
if (strcmp(s1, "login") == 0)
printf("s1 = \"login\"\n");
else
printf("s1 != \"login\"\n");
return 0;
}
```

Backup: Let's...

Original slide 391

- Kahoot time!
- Q1 - Q10



Files

Files

Live pacing: about 8 min

Question. How does C handle text and external data safely?

Core claim. String and file APIs are powerful only when sizes, terminators, and error states are checked.

Target capability. Students can choose bounded string/file operations and test failure paths.

Main Path

- Start from the question: How does C handle text and external data safely?
- Build the model: String and file APIs are powerful only when sizes, terminators, and error states are checked.
- End with a checkable action: Students can choose bounded string/file operations and test failure paths.
- Keep only this slide on the horizontal path during the live lecture.

Worked Example

- Use this as the live trace: Read a line with a fixed buffer, detect truncation/newline, then write parsed output to a file.
- Representative source slide: 392
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Anecdote Or Motivation

In C, input is hostile by default: it may be too long, malformed, missing, or late.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

- Example

```
#include <stdio.h>
#include <string.h> /* strlen() */
#include <stdlib.h> /* exit() */
int main() {
FILE *file;
char *msg = "123456789012345";
int n;
file = fopen("tmp/test_write", "w");
if (file!= NULL) {
n = strlen(msg);
int res = fwrite(msg, sizeof(char), n, file);
if (res!=n) {
fprintf(stderr,"Error during writing\n");
exit(2);
}
fprintf(stdout,"Write successful\n");
fclose(file);
}else{
printf("Couldn't open file for writing\n");
}
return 0;
}
```

#400

Backup Index

Original slides 392-400

I/O and files

In C, the I/O are implemented using files
There are a lot of predefined files
The standard input stdin
The standard output stdout
The standard error stderr
We have already seen the basic formatted I/Os
printf(...): write to the file of the stdout
scanf(...) : read from the file stdin
More global I/Os exist
Non formatted
Based on real files

Backup: I/O and files

Original slide 392

```
In C, the I/O are implemented using files
There are a lot of predefined files
The standard input stdin
The standard output stdout
The standard error stderr
We have already seen the basic formatted I/Os
printf(...): write to the file of the stdout
scanf(...) : read from the file stdin
More global I/Os exist
Non formatted
Based on real files
```

Backup: File declaration

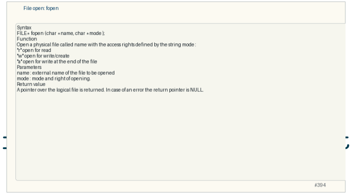
Original slide 393

```
The access to the files is performed through logica
We have to use the directive #include <stdio.h>
We define them as type FILE
We use pointer to access them FILE*
Example of declaring a logical file
FILE* file;
Remark:
A logical file of a program has to be linked with an existing physical file
The link is created through the operations of opening and closing the logica
```

Backup: File open: fopen

Original slide 394

```
Syntax
FILE* fopen (char *name, char *mode );
Function
Open a physical file called name with the access ri rin
"r" open for read
"w" open for write/create
"a" open for write at the end of the file
Parameters
name : external name of the file to be opened
mode : mode and right of opening.
Return value
A pointer over the logical file is returned. In case of an error the return
```



File open: fopen

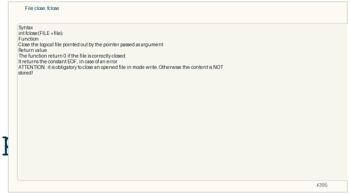
Syntax
FILE* fopen (char *name, char *mode);
Function
Open a physical file called name with the access rights defined by the string mode :
"r" open for read
"w" open for write/create
"a" open for write at the end of the file
Parameters
name : external name of the file to be opened
mode : mode and right of opening.
Return value
A pointer over the logical file is returned. In case of an error the return pointer is NULL.

#394

Backup: File close: fclose

Original slide 395

```
Syntax
int fclose(FILE *file);
Function
Close the logical file pointed out by the pointer p
Return value
The function return 0 if the file is correctly closed
It returns the constant EOF , in case of an error
ATTENTION : it is obligatory to close an opened file in mode write. Otherwis
```


File close: fclose

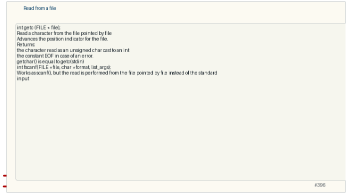
Syntax
int fclose(FILE *file);
Function
Close the logical file pointed out by the pointer passed as argument
Return value
The function return 0 if the file is correctly closed
It returns the constant EOF , in case of an error
ATTENTION : it is obligatory to close an opened file in mode write. Otherwise the content is NOT stored!

#395

Backup: Read from a file

Original slide 396

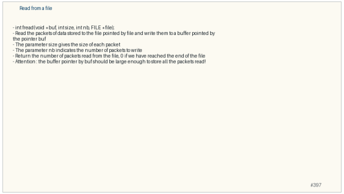
```
int getc (FILE * file);
Read a character from the file pointed by file
Advances the position indicator for the file.
Returns:
the character read as an unsigned char cast to an i
the constant EOF in case of an error.
getchar() is equal to getc(stdin)
int fscanf(FILE *file, char *format, list_args);
Works as scanf(), but the read is performed from the file pointed by file in
```



Backup: Read from a file

Original slide 397

- int fread(void *buf, int size, int nb, FILE *file);
- Read the packets of data stored to the file pointed by file and write them to a buffer pointed by the pointer buf
- The parameter size gives the size of each packet
- The parameter nb indicates the number of packets to write
- Return the number of packets read from the file, 0 if we have reached the end of the file
- Attention : the buffer pointer by buf should be large enough to store all the packets read!


Read from a file

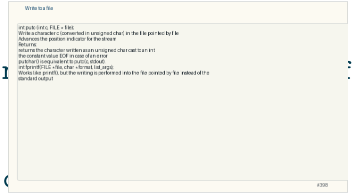
- int fread(void *buf, int size, int nb, FILE *file);
- Read the packets of data stored to the file pointed by file and write them to a buffer pointed by
the pointer buf
- The parameter size gives the size of each packet
- The parameter nb indicates the number of packets to write
- Return the number of packets read from the file, 0 if we have reached the end of the file
- Attention : the buffer pointer by buf should be large enough to store all the packets read!

#397

Backup: Write to a file

Original slide 398

```
int putc (int c, FILE * file);
Write a character c (converted in unsigned char) i           file
Advances the position indicator for the stream
Returns:
returns the character written as an unsigned char c
the constant value EOF in case of an error
putchar() is equivalent to putc(c, stdout).
int fprintf(FILE *file, char *format, list_args);
Works like printf(), but the writing is performed into the file pointed by f
```



Write to a file

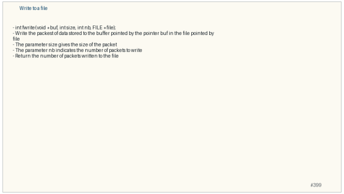
int putc (int c, FILE * file);
Write a character c (converted in unsigned char) in the file pointed by file
Advances the position indicator for the stream
Returns:
returns the character written as an unsigned char cast to an int
the constant value EOF in case of an error
putchar() is equivalent to putc(c, stdout).
int fprintf(FILE *file, char *format, list_args);
Works like printf(), but the writing is performed into the file pointed by file instead of the standard output

#398

Backup: Write to a file

Original slide 399

- int fwrite(void *buf, int size, int nb, FILE *file);
- Write the packest of data stored to the buffer pointed by the pointer buf in the file pointed by file
- The parameter size gives the size of the packet
- The parameter nb indicates the number of packets to write
- Return the number of packets written to the file



Backup: Example

Original slide 400

```c
#include <stdio.h>
#include <string.h> /* strlen() */
#include <stdlib.h> /* exit() */
int main() {
FILE *file;
char *msg = "123456789012345";
int n;
file = fopen("tmp/test_write", "w");
if (file!= NULL) {
n = strlen(msg);
int res = fwrite(msg, sizeof(char), n, file);
if (res!=n) {
fprintf(stderr,"Error during writing\n");
exit(2);
}
```